

VERIQO -- VERIFICATION & TRUST BOUNDARY RESPONSE

Assessment date: 29 August 2026

Baseline: Red Flag Response Report -- commit e0dcdbb

Responds to: Verification.docx -- an independent reviewer's follow-up review of the Red Flag Response, agreeing with the direction taken but identifying that "unexported fields" alone does

not prove perfect immutability, that hash-consistency alone does not prove authoritative lineage,

and proposing two structural changes: a 4-level maturity model (replacing the prior 3-level one)

and a repository-wide Authority Boundary Audit as the next phase, explicitly not "adding features."

Assessment mode: local engineering qualification and reproducibility review. All code in this

deliverable compiles, is tested, and was verified against the live repository. Every claim below

that names a test, a file, or a line of code was checked against that exact source before this

report was written.

EXECUTIVE VERDICT

Every item the reviewer raised is addressed below, in the reviewer's own order. Three are real code

gaps that got real fixes, each verified with the same "revert the fix, prove the test fails, restore the fix, prove the test passes" discipline used throughout this engagement:

1. AuthorizedFinding was not actually immutable -- unexported fields stop cross-package *construction*, not cross-package *mutation* of what is already sealed, because Go copies a struct's slice fields by header, not by cloning the backing array. Fixed with defensive deep-copying at both the write boundary (Authorize) and the read boundary (Finding()).
2. Authorize() verified hash-consistency, not full lineage -- a Finding whose hash verified could still cite evidence that was never real, and could cite a hypothesis whose own Status was never actually evidence-derived. Both gaps are closed: AuthorizeGrounded verifies every cited evidence ID resolves to a real, FINALIZED, hash-verified manifest.Manifest; a new check in VerifyFindingAgainstHypothesis refuses any hypothesis whose stored Status exceeds what its own evidence lists could legitimately derive.
3. No test proved authority survives the whole pipeline -- TestAuthorityCannotBeLostAcrossThePipeline now runs Detection -> Inference -> Finding -> Authorization -> AuthorizedFinding -> Decision ->

Reporting end to end with real packages at every stage, and proves the forbidden shape (AuthorizedFinding -> cast/rebuild -> Raw Finding -> Decision) is refused at the authorization

boundary, not merely absent from the code someone happened to write.

The reviewer's exact required language for the inference engine and for LocalAnchorer is adopted

verbatim below (see "Calibrated status language"), the 4-level maturity model replaces the prior

3-level one, and the Authority Boundary Audit -- explicitly framed by the reviewer as classification, not a feature-development mandate -- is completed for every named object, with one

additional real fix (the hypothesis-status gap above) where the audit surfaced a genuine, narrowly scoped bypass.

PART 1 -- DEEP IMMUTABILITY OF AUTHORIZEDFINDING

Reviewer's point: "Kalimat Claude: 'semua field unexported, tidak bisa dibuat dari package lain' belum otomatis berarti sistemnya immutable secara sempurna" -- unexported fields prevent construction, not necessarily mutation, if an accessor returns a slice/map/pointer directly. Nested objects (slices, maps, pointers, nested structs, interfaces, returned references) all need auditing, not just the top-level struct.

Response: The reviewer's critique was correct and identified a real, exploitable bug, not a theoretical one. finding.Finding has three []string fields -- SupportedBy, ContradictedBy, Alternatives -- and Go copies a struct's slice field by header (pointer, length, capacity), never by cloning the backing array. AuthorizedFinding.Finding() returned a finding by value, which looks like a safe copy but still shared the same three backing arrays with a's own sealed state. A caller mutating an element of the returned Finding's SupportedBy slice was mutating AuthorizedFinding's own internal data out from under its already-computed AuthorizationHash, silently, with no error anywhere. The same hole existed in the other direction: a caller could mutate the Finding they built *after* calling Authorize, corrupting the sealed value retroactively.

Fix (pkg/insurance/cre/authorized.go): a cloneFinding helper deep-copies all three slice fields -- the only reference-type fields finding.Finding has; every other field is a scalar, so a full audit of nested slices/maps/pointers/interfaces reduces to exactly these three. It is applied at both boundaries: Authorize clones before sealing f into the new AuthorizedFinding, and Finding() clones again before returning. Every call to Finding() now returns a fully independent copy in both directions.

Verified as exploitable, then verified as fixed. Two new tests -- TestFindingAccessorReturnsAnIndependentCopy and TestAuthorizeClonesInputSoCallerCannotMutateAfterTheFact -- were first run against the pre-fix code (the clone calls temporarily reverted) and both failed with exact corruption evidence (got [TAMPERED ev-weather-log-1], want [ev-survey-1 ev-weather-log-1]). Restoring the fix, both tests pass, and the full pkg/insurance/cre suite (31 tests) remains green.

PART 2 -- AUTHORIZE() MUST VERIFY FULL LINEAGE, NOT JUST HASH CONSISTENCY

Reviewer's point: "Hash membuktikan: 'data ini konsisten dengan hash.' Hash tidak membuktikan: 'data ini berasal dari authoritative source.'" A Finding's lineage -- hypothesis ID, hypothesis content, evidence references, evidence hashes, confidence basis, generation context, policy context -- must be traceable, not just internally self-consistent.

Response: Two independent gaps existed under this one heading, and both are now closed.

Gap 2a -- evidence references were never checked against real evidence. Authorize already confirmed (via VerifyFindingAgainstHypothesis) that a Finding's SupportedBy/ContradictedBy match the real hypothesis's own evidence *references* -- but a reference is just a string ID. Nothing previously confirmed that ID names evidence that actually exists, let alone evidence whose

own integrity independently verifies. `AuthorizeGrounded` (`pkg/insurance/cre/authorized.go`) closes this by reusing the already-existing `pkg/evidence/manifest` package (no new evidence-authenticity system invented): every evidence ID a Finding cites in `SupportedBy` or `ContradictedBy` must resolve, in a supplied `manifest.Registry`, to a `manifest.Manifest` whose latest version is `FINALIZED` and whose own hash independently verifies via `manifest.VerifyManifestHash`. Five new tests cover acceptance of real finalized evidence and refusal of a nil registry, unknown evidence, non-finalized evidence, and partially grounded evidence (one real citation, one fabricated).

Gap 2b -- a hypothesis's own Status was never confirmed to be evidence-derived. `causation.HypothesisSet.Add` does not force Status to be computed from real evidence -- a caller can Add a Hypothesis whose Status already claims `StatusSupported` while its own `SupportingEvidence/ContradictingEvidence` lists would legitimately derive something weaker (or nothing at all), and Add accepts it as long as the value is a known `causation.Status`. Since `VerifyFindingAgainstHypothesis` only compared `f.ConfidenceBasis` against `h.Status`, a Finding citing such a hypothesis passed even though `h.Status` was never actually earned -- exactly the "Hash proves consistency, not authoritative origin" distinction the reviewer named. Fixed in `pkg/insurance/cre/provenance.go`: `VerifyFindingAgainstHypothesis` now also recomputes the status `h`'s own evidence lists would produce via `causation.DeriveStatus(h, nil)` -- already exported by the mature causation package for exactly this purpose, so that package needed no changes -- and refuses whenever `h.Status` is strictly stronger than that recomputation. The comparison is deliberately one-directional: `DeriveStatus(h, nil)` counts evidence with no independence discounting, the most generous status any legitimate computation could produce, so a real dependency-graph-discounted status can only be weaker or equal, never stronger -- this check cannot false-positive against any legitimate computation anywhere in the codebase. New test `TestVerifyFindingAgainstHypothesisRejectsHypothesisWithFabricatedStatus` proves the exact attack is caught, through both the standalone check and the full `Authorize` gate, verified failing against the pre-fix code before confirming it passes with the fix.

Every other lineage element the reviewer listed was already covered before this round and remains covered: hypothesis ID and content (`VerifyFindingAgainstHypothesis` re-derives from the real `HypothesisSet`, never trusts the Finding's own claim), confidence basis (same), generation context (`VerifyFindingProvenance` confirms a cited `SourceInferenceTraceID` names a real, hash-verified `InferenceTrace`), and policy/model context (the trace's own model lineage runs through `pkg/governance/lifecycle`'s `Registered` -> `Validated` -> `Calibrated` -> `Approved` -> `Active` gate, exercised end to end in Part 3's pipeline test).

PART 3 -- REDFLAG-2 PIPELINE SECURITY TEST, STRENGTHENED

Reviewer's point: prove not just `GenerateFindings()` -> `authorized`, but the full chain -> downstream processing -> cannot downgrade -> cannot reconstruct raw Finding -> cannot bypass authorization, across `Detection` -> `Inference` -> `Finding` -> `Authorization` -> `AuthorizedFinding` -> `Decision` -> `Reporting`, explicitly forbidding `AuthorizedFinding` -> `cast/rebuild` -> `Raw Finding` -> `Decision`.

Response: TestAuthorityCannotBeLostAcrossThePipeline
(test/integration/authority_pipeline_test.go) is new. It builds a real pipeline end to end:

- Detection -- a real ontology.Registry object, two real manifest.Manifest records (one supporting, one contradicting) taken all the way to FINALIZED with an independently verifying hash.
- Inference -- a real lifecycle.Registry model taken to ModelActive through its full governance gate, then a real inference.Recorder.Record call producing a hash-verified trace.
- Finding -- a real causation.HypothesisSet, with both supporting and contradicting evidence attached so the hypothesis's real status is PARTIALLY_SUPPORTED, not trivially SUPPORTED.
- Authorization -- both cre.GenerateFindings (the production path) and the stricter cre.AuthorizeGrounded succeed against this real data.
- Decision & Reporting -- two stand-in functions, decide and report, typed to accept only cre.AuthorizedFinding, never finding.Finding. This typing *is* the proof for the forbidden diagram: raw := authorized.Finding(); decide(raw) does not compile, because finding.Finding is not cre.AuthorizedFinding -- Go has no runtime "must not compile" primitive, so the test file records this as a documented, verified fact in its own comment rather than asserting it at runtime.
- Adversarial re-entry attempt -- the test extracts af.Finding() (an independent copy, per Part 1's fix), escalates its ConfidenceBasis to a status the real hypothesis did not earn, recomputes the Finding's own hash to stay internally self-consistent (exactly what an attacker capable of forging a Finding would also do), and confirms both cre.Authorize and cre.AuthorizeGrounded refuse to re-authorize it. The original AuthorizedFinding's own state -- and decide(af)'s output -- is confirmed unaffected by any of this.

This is the strongest form of the requirement available without a language-level "must not compile" test primitive: a real, running, end-to-end positive path, plus a compile-time-enforced negative path recorded and verified as a fact about the type system, plus a runtime proof that re-injection is refused by the same gate a first-time forgery would hit.

PART 4 -- CALIBRATED STATUS LANGUAGE

The reviewer asked that two specific claims never be overstated, and supplied the exact replacement language. Both are adopted verbatim, here and in every future report from this engagement.

Inference engine. Never: "VERIQO AI inference engine production-ready." Always:

> VERIQO has a governed inference architecture and executable integration path, but external model
> execution remains environment/provider dependent and has not yet been production-validated.

Model Registry -> Approval -> Policy -> Evidence Grounding -> Inference Governance -> Output Validation

-> Human Review -> Audit is tested as an integration architecture (Part 3 above, and the prior Red Flag Response's TestModelGatewayFullPipeline). Actual external model execution against a real provider is not, and this report makes no claim that it is.

LocalAnchorer. The reviewer's own four-row framing, kept explicitly separated rather than collapsed into a single "done" label:

Layer Status

--- ---

Engineering [GREEN] Implemented
Internal verification [GREEN] Tested
External anchoring [RED] Not live
Independent third-party verification [RED] Not proven

LocalAnchorer's own package doc comment and every AnchorReceipt.AnchoredBy value it produces already say "LocalAnchorer(simulator, not a real external anchor)" verbatim -- no code changed here, only this report's framing, so the distinction stays visible rather than living only in a code comment.

PART 5 -- "57 TESTS PASSING" IS NOT 57 PROOF POINTS

The reviewer's principle is adopted explicitly, and is restated here so it travels with every future status claim this engagement makes:

> Software correctness != Domain truth != Real-world truth.

A test suite proves the code behaves as the test cases we wrote say it should. It does not prove:
real maritime data correctness, real AIS anomaly correctness, real cargo identity, real owner identity, real insurance coverage, real P&I response, real trade finance workflow, real claims, real counterparties, real sanctions environment, real-world legal enforceability, or real external anchoring. Every "tests passing" claim in this and prior reports should be read against this principle, not in place of it.

PART 6 -- FOUR-LEVEL MATURITY MODEL (REPLACES THE PRIOR THREE-LEVEL MODEL)

The prior Red Flag Response adopted a 3-level framework (Code Complete / Engineering Verified / Real-World Production Verified). The reviewer's critique is accepted: VERIQO's trust architecture is more complex than that framework distinguishes, specifically between a *controlled* pilot with real external parties and *unsupervised production* exposure. The reviewer's 4-level model replaces it, effective immediately, in this and all future reports:

- LEVEL 1 -- Implemented. Code exists: code, build, basic tests.
- LEVEL 2 -- Engineering Verified. Unit, integration, race, vet, guardrails, determinism, replay, fault tests.
- LEVEL 3 -- Controlled External Validation. Real data, real documents, real counterparties, real workflows, real domain experts -- but still a controlled pilot.
- LEVEL 4 -- Production Proven. Real customers, real decisions, real incidents, real claims, real financial impact, real SLA, real security, real external integrations.

VERIQO's honest position on this axis, as of this report: Level 2. Everything in this deliverable -- the immutability fix, the lineage-grounding gate, the hypothesis-status check, the pipeline test, the full repository test suite (go build, go vet, gofmt, the full go test ./..., -race on the affected packages, and pkg/insurance/guardrails) -- is Level 2 evidence.

None of it is Level 3 or Level 4 evidence, and this report makes no claim that it is.

"Production

proven" is reserved for Level 4, exactly as the reviewer specified, and is not used anywhere in this report.

PART 7 -- AUTHORITY BOUNDARY AUDIT

Reviewer's framing, quoted directly: *"Bukan langsung menambah fitur. Tetapi lakukan Authority*

Boundary Audit." (Not adding features directly. Instead perform an Authority Boundary Audit.) The

task: search the codebase for every object carrying a settable boolean verified / trusted / approved / authorized / reconciled / settled / insured / compliant pattern (or an equivalent enum

field) that can be set directly by a caller -- the Thing{Verified: true} trust-bypass shape

-- and

classify each: Object / Forgeable? / Gate / Authority Source / Status.

Method. Two passes: (1) a repository-wide search for exported struct fields of type bool, Status, or State whose name matches Verified|Trusted|Approved|Authorized|Reconciled|Settled|Insured|Compliant|Validated|Confirmed|Final|Immutable|Canonical; (2) for every hit, tracing whether the field is a derived, computed report/output value (safe -- nothing downstream trusts

it as an input) or a directly caller-settable authority value with no gating computation behind

it (the actual bypass pattern). The reviewer's own named object list is covered below; two names on

that list -- InsuranceAssessment and RiskAssessment -- do not exist as named types anywhere in

this repository, and are reported as such rather than invented for the sake of a complete-looking table.

Object	Forgeable?	Gate	Authority Source	Status
--------	------------	------	------------------	--------

---	---	---	---	---
-----	-----	-----	-----	-----

Finding	No	Authorize() / AuthorizeGrounded(), both requiring a real causation.HypothesisSet and (for the grounded gate) a real manifest.Registry cre.AuthorizedFinding -- unexported fields, sealed by Authorize, deep-cloned at both boundaries (Part 1)	[GREEN]	Closed
---------	----	--	---------	--------

Hypothesis	No	(as of this round) VerifyFindingAgainstHypothesis's new statusStrength / causation.DeriveStatus(h, nil) check causation.HypothesisSet, evidence-derivation now enforced one-directionally	[GREEN]	Closed this round (Part 2b)
------------	----	---	---------	-----------------------------

Evidence	Yes	evidence.Registry.SetStatus(evidenceID, status) accepts any known Status value from any caller, with no verification-computation gate behind it none -- the caller's own claim	[RED]	Open, documented, not fixed this round -- see below
----------	-----	--	-------	---

Decision (pkg/moat/decision)		Constructible, but not currently exploitable decision.Decision and DecisionRecord are plain exported structs; only Engine.Decide is exercised anywhere in this codebase as a real decision path, and DecisionRecord's audit log is hash-chained and append-only, produced only internally Engine.Decide, when used	[YELLOW]	Narrow -- see below
------------------------------	--	--	----------	---------------------

Claim (pkg/insurance/claim)		Constructible, but narrow blast radius claim.New only sets the initial StatusRegistered value; nothing in the claim package gates later transitions of Claim.Status case.Case.RegisterClaim checks Status == StatusRegistered once, at registration only; no other code in the repository reads Claim.Status to gate a money-movement or settlement decision	[YELLOW]	Narrow -- see below
-----------------------------	--	--	----------	---------------------

Settlement (pkg/insurance/payment SettlementEvidence / SettlementReconciliation)	No	PaymentRecord.RecordSettlementEvidence -- unexported p.settlement field, requires StatusPaid, validates the evidence, refuses to overwrite an existing record		
--	----	---	--	--

PaymentRecord, itself state-machine gated [GREEN] Already sealed
Counterparty (closest existing analog: pkg/insurance/party.Party) No trust-bypass
field found n/a -- Party carries no Verified/Trusted/Approved boolean at all n/a
[GREEN] Nothing to close
EntityResolution (closest existing analog: pkg/moat/entity.Registry) No CanonicalID
is content-addressed (SHA-256 over the sorted alias set), never caller-settable; every
MergeRecord is hash-chained, append-only, and produced only by Registry.Merge
entity.Registry's own hash chain [GREEN] Already sealed
InsuranceAssessment -- -- -- Not found -- no type of this name exists in this
repository; not invented for this table
RiskAssessment -- -- -- Not found -- no type of this name exists in this
repository; not invented for this table

Notes on the three non-[GREEN] rows.

Evidence -- genuinely open. evidence.Registry.SetStatus is a real, exploitable trust-
bypass
pattern: any caller can set an evidence record's Status to any known value, including
StatusVerified, with no computation behind it. This is the same category of gap Part 2b
closed
for Hypothesis, but in a larger, more mature subsystem (pkg/insurance/evidence, unchanged
since
early in this engagement) with a wider blast radius, so closing it correctly needs its own
scoped
round rather than a rushed fix inside this one -- matching the reviewer's own framing that
this
phase is audit-and-classify, not "fix everything found." It is recorded here honestly as the
one
[RED] finding, not silently left off the table.

Decision -- constructible but not currently a live bypass. A caller can write
decision.Decision{Action: decision.ActionEscalate} directly; nothing stops it. But no code
anywhere in this repository currently *consumes* a bare, caller-constructed Decision as an
input
to a further gated action -- the only place Decision values are produced is Engine.Decide's
own
internal computation, and the audit log (DecisionRecord) is hash-chained and only appended
to
internally. This is a latent risk (a future caller *could* build a fake Decision and hand it
somewhere that trusts it) rather than a live one today, and is recorded as such rather than
either
ignored or overstated as an active exploit.

Claim -- same shape as Decision, verified by tracing every reader of Claim.Status. Only
one place
in the repository reads it: case.Case.RegisterClaim's one-time entry guard. No coverage,
quantum, recovery, or payment code path keys a decision off Claim.Status. The real
settlement
authority gate is pkg/insurance/payment's state-machine-backed PaymentRecord, already sealed
(see the Settlement row above), so Claim.Status being directly settable does not currently
open a
path to an unauthorized payment. Recorded as [YELLOW] rather than [GREEN] because the field
itself remains
unsealed, even though nothing today exploits that.

VERIFICATION EVIDENCE

- gofmt -l . -- clean, no output.
- go build ./... -- clean.

- go vet ./... -- clean.
- go test ./... -- full repository suite, all packages pass (including test/integration, test/e2e/eight_blockers, test/soak, test/stress, test/acceptance).
- go test -race ./pkg/insurance/cre/... ./test/integration/... ./pkg/insurance/causation/... ./pkg/insurance/evidence/... -- clean, no data races.
- go test ./pkg/insurance/guardrails/... -- all 7 repository-wide guardrail scans pass (no Determination field, no opaque confidence score, no forbidden canonical duplicate, no hard-coded vendor judgment, anywhere in the insurance domain).
- pkg/insurance/cre test count: 31, all passing (up from 23 before this round: 2 new immutability tests, 5 new grounding tests, and 1 new hypothesis-status-fabrication test).

HONEST SCOPE BOUNDARY FOR THIS ROUND

Per the reviewer's own instruction that this phase is an audit, not a feature mandate: the Authority Boundary Audit above classifies every named object and fixes the one gap (Hypothesis.Status) that was both (a) directly reachable from the Finding/Authorize chain this engagement has been hardening and (b) closeable with a small, reuse-only change to already-exported API, exactly like Part 2b. The Evidence.SetStatus gap is real, is not fixed here, and is not described as fixed anywhere in this report. Closing it -- along with re-auditing Decision and Claim once (or if) real callers start consuming their currently-unconsumed authority fields -- is the next scoped round's work, not claimed as done in this one.